CONCISE INTUITIONISTIC ARITHMETIC

One computation axiom for iPRA

A six-rule presentation with zero-normal terms

A checked simplification of the current ConciseIPRA proposal

Result. Make only $I(0, a; u.h) = a$ definitional. Use a term layout in which the initializer a stays outside the state binder. The existing single-introduction calculus then needs six primitive rules, including only one computation schema. The complete, strictly validating tagged Python checker decreases from 3,145 to 3,095 normalized bytes, from 1,024 to 1,013 significant tokens, and from 1,080 to 1,075 AST nodes. The normalization and its input checks are included.

Equivalence. The new theory is bi-interpretable and equiconsistent with full first-order intuitionistic primitive recursive arithmetic. The numerical domains and equality are unchanged. Both interpretation composites are internally the identity; on old term syntax this uses $t = \mathrm{nf}(t)$.

Scope. This is a local reduction with a written proof, a working checker, proof translations, and compiled Lean verification of the normalization bridge. The entire six-rule equivalence is not formalized in Lean. The construction does not beat the repository's 521-byte arity-coded checker and does not establish an absolute minimum. Earlier packages explicitly proposed the underlying idea; the present contribution makes it work at lower cost in the direct tagged layout.

Prepared for Vladimir Reshetnikov

11 November 2026

Baseline: ProveIt commit
`ecb3b847290c33b1ef7f04571cac63e97c73a54e`

Abstract

The native ConciseIPRA calculus uses numerical iterator terms, equations, and two binders Q, D. Its current reductions already remove separate logical axioms and combine the case and induction introductions. We complete an earlier suggested reduction: use the normal forms of the single erasing rewrite $I(0, a; u.h) \to a$ as terms. Hereditary substitution then absorbs zero computation, leaving successor computation as the sole nonlogical axiom schema. A different closed representative of zero, $K = I(1, 0; u.0)$, preserves the single-introduction construction. We give all six rules, effective translations in both directions, both round trips, and the transfer of the existing full iPRA bi-interpretation. The changed syntax, scope validation, and hereditary substitution are charged in a matched implementation comparison.

Contents

1 What changes, and what was already known **2**

2 Term syntax and hereditary substitution **2**

3 The complete six-rule theory **3**

4 Recovering equality and both introductions **4**

5 Normalization algebra **6**

6 The internal round trip **6**

7 Both proof translations **7**

8 Bi-interpretability with full iPRA **8**

9 The standalone checker **9**

10 Size comparison and limits **10**

11 Verification and formalization scope **11**

12 Reproduction and contents **12**

A The inherited iPRA interface **13**

B A scope example **13**

1 What changes, and what was already known

The comparison is to the current repository at the commit on the title page, including its fourth delivery of research packages. The native kernel has eight constructors: hypothesis, two computation equations, equational transport, Q elimination, D elimination, D introduction by cases, and equational Q introduction by progress. The existing `witness_rule/NativeOneIntro` development combines the last two introductions. The existing `one_context` proposal replaces two-template equality transport by one crossed numerical context [1, 2, 3, 4].

The present change is

Remove zero computation from proofs; normalize it in term syntax.

It helps only if the normalization costs less than the deleted proof-checking case. Section 10 measures the complete cost, including the canonical-term check.

1.1 Direct antecedents

Definitional zero computation is not a new mathematical idea. The subsection *Making zero computation definitional* in `prevalidated_elimination/theory/research.tex` already proposes it and explains why the examined implementation lost: its arity codec places an initializer under a dummy slot, so erasing an iterator requires lowering coordinates. `uniform_zero/report.tex` explicitly asks whether a different binder layout could make the conversion cheaper [5, 6].

The direct tagged syntax has that different layout. In $I(c, a; u.h)$, only h binds u; c, a remain at the surrounding scope. Erasing a zero-counter iterator returns a without scope adjustment. This report supplies the smaller implementation, the revised witness construction, and the proof obligations for both interpretation composites.

1.2 The exact simplicity claim

The result has two independently checkable parts:

1. The complete axiomatization has six primitive proof rules and only one computation axiom schema.
2. Against an equally factored seven-rule tagged control, the standalone checker is smaller in normalized bytes, significant tokens, and AST nodes. Both programs validate numerical scopes and formula sorts. The public term grammars differ intentionally.

The 521-byte programs use a different arity-coded protocol, guarded public language, and auxiliary resources. They remain smaller programs. No global lower bound, absolute optimum, or new record over that frontier is claimed.

2 Term syntax and hereditary substitution

2.1 Raw terms and normal forms

Start with raw terms

$$t ::= x \mid 0 \mid S(t) \mid I(c, a; u.h),$$

where only u is bound, and only in h. There is one numerical sort. Define the bottom-up operation

$$\mathrm{nf}(x) = x, \qquad \mathrm{nf}(0) = 0, \quad \mathrm{nf}(St) = S\,\mathrm{nf}(t),$$
$$\mathrm{nf}(I(c,a;u.h)) = \begin{cases} \mathrm{nf}(a), & \mathrm{nf}(c) \text{ is literally } 0, \\ I(\mathrm{nf}(c), \mathrm{nf}(a); u.\,\mathrm{nf}(h)), & \text{otherwise.} \end{cases}$$

New terms are exactly the fixed points of nf: well-scoped terms with no literal-zero-counter iterator at any depth. For already normal arguments define the smart constructor

$$\widehat{I}(c,a;u.h) = \begin{cases} a, & c \equiv 0, \\ I(c,a;u.h), & c \not\equiv 0. \end{cases} \tag{SC}$$

Here $\equiv$ is literal syntax. This constructs syntax; it adds no object-language arithmetic function or axiom.

Only zero computation is normalized. In particular

$$K := I(S0, 0; u.0) \tag{K}$$

is a normal closed term distinct in syntax from 0. Its equality to zero is proved using the remaining axiom.

2.2 Substitution and formulas

All substitutions are capture-avoiding substitutions followed by normalization:

$$t[\sigma]_0 := \mathrm{nf}(t[\sigma]). \tag{HS}$$

Compute this directly by recursively rebuilding with $\widehat{I}$. Substitute in the iterator counter and initializer at the current scope, and use the lifted substitution only in its body. Renaming preserves normality and is ordinary renaming.

Formulas are

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A.$$

The two binders bind the displayed numerical variables. In de Bruijn notation y has index 0 and x index 1 in a D-matrix. Formula substitution normalizes its numerical terms and lifts under each binder. Normalization never changes a formula's root constructor.

Every $B(t,r)$ below means simultaneous hereditary instantiation of $B(x,y)$, and $h(t)$ means $h[t/u]_0$. Bound variables are renamed fresh before substitution. Neither the normalizer nor the introduction rule tests whether an arbitrary term *denotes* zero.

3 The complete six-rule theory

Let $\mathcal{Z}_6$ have finite lists Γ of reusable assumptions. Its designated contradiction is $0 = S0$. These are all six primitive rules; IS is a zero-premise axiom schema.

1. Hypothesis.

$$\frac{}{\Gamma \vdash A} \quad (A \in \Gamma). \tag{Hyp}$$

2. Successor computation. For arbitrary normal c, a, h,

$$\frac{}{\Gamma \vdash I(Sc, a; u.h) = h[\widehat{I}(c, a; u.h)/u]_0}. \tag{IS}$$

The left side is normal because Sc is not literal zero.

3. One-context crossed equality. For a normal numerical template $h(v)$,

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash h(s) = r}{\Gamma \vdash r = h(t)}. \tag{X}$$

There is no restriction on s.

4. Closure elimination.

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}. \tag{QE}$$

5. Witness elimination.

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x, y) \qquad \Gamma, \mathsf{Q}_y B(b, y) \vdash A}{\Gamma \vdash A}. \tag{DE}$$

The eigenvariable b is fresh for Γ, A and the fixed matrix parameters. It is not a freely chosen witness term.

6. Single introduction.

$$\frac{\Gamma, B(t, \eta) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}, \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases} \tag{DI}$$

The variable z is fresh for Γ, t and the fixed parameters of B. The displayed local assumption is discharged.

There are no additional primitive reflexivity, symmetry, substitution, cut, induction, successor, coverage, separation, or logical axioms. The ordinary readings established by the interpretation are

$$\mathsf{Q}_y B(y) \rightsquigarrow B(0) \to \forall y\, B(y), \qquad \mathsf{D}_{x,y} B \rightsquigarrow \exists x \big(B(x, 0) \to \forall y\, B(x, y)\big). \tag{Reading}$$

These readings explain the binders and do not add proof rules.

4 Recovering equality and both introductions

4.1 Reflexivity without zero computation

Lemma 4.1. *For every normal term r, $\mathcal{Z}_6$ proves $r = r$.*

Proof. IS with $c = 0, a = 0$ and the constant body r, weakened under the state binder, gives $L_r = r$, where $L_r = I(S0, 0; u.r)$. Use this equation as both premises of X with identity template. The result is $r = r$. □

Symmetry follows from X with major $s = s$, minor $s = t$, and identity template. Given $r = s$ and $s = t$, reverse the first equation and apply X with major $s = t$, minor $s = r$, and the identity template; its conclusion is $r = t$. Congruence follows from X with major $s = t$ and reflexive minor $h(s) = h(s)$. Two X steps recover the full two-template transport:

$$\begin{aligned} s = t,\ a(s) = b(s) &\implies b(s) = a(t), \\ s = t,\ b(s) = a(t) &\implies a(t) = b(t). \end{aligned} \tag{Tr}$$

4.2 Structural facts

Weakening, exchange, contraction, variable renaming, and ordinary assumption cut are admissible by direct induction on a proof. Weakening and renaming preserve the formula root and the literal-zero status of a DI witness. For cut, replace uses of the discharged assumption by its proof, renaming eigenvariables apart. This does not substitute a numerical term into the witness.

Arbitrary numerical substitution of six-rule proofs is established later through a two-introduction calculus. It cannot be justified by an unqualified constructor induction: substitution may turn a nonliteral witness into literal zero.

4.3 A seven-rule bridge

Define $\mathcal{Z}_7$ on the same normal syntax by replacing DI with

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)} \quad (\mathrm{DI}_{\mathrm{case}})$$

for arbitrary matrices and witnesses, and

$$\frac{\Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash \mathsf{Q}_y E(y)} \quad (\mathrm{QI}_{\mathrm{prog}})$$

for literal equations only. The corresponding freshness conditions are retained. These seven rules are stable under numerical substitution.

Theorem 4.2. *$\mathcal{Z}_6$ and $\mathcal{Z}_7$ derive exactly the same sequents.*

Proof. To recover case introduction in $\mathcal{Z}_6$, only an equation $E(x,y)$ at witness 0 is missing. Given

$$\Gamma, E(0,0) \vdash E(0,Sz),$$

use $K = I(S0, 0; u.0)$. It is normal and syntactically nonzero, and IS proves $K = 0$. Under $E(K,0)$, equational transport gives $E(0,0)$. Cut in the supplied proof and transport its conclusion to $E(K,Sz)$, using $0 = K$. DI at K selects case mode and concludes the desired $\mathsf{D}_{x,y}E$. This uses equation transport and proof-cut, not an assumed numerical substitution theorem.

For progress introduction of $E(y)$, add an unused witness x and use DI at witness 0. It gives $\mathsf{D}_{x,y}E(y)$. DE exposes exactly the desired $\mathsf{Q}_y E(y)$, independent of its fresh witness; Hyp returns it.

Conversely, a case-mode DI is the case rule. For a progress-mode DI, its matrix is an equation and its witness is zero. $\mathrm{QI}_{\mathrm{prog}}$ gives $\mathsf{Q}_y B(0,y)$. Under $B(0,0)$, QE gives $B(0,Sz)$; case introduction at zero then produces $\mathsf{D}_{x,y}B$. □

Corollary 4.3 (Equation induction)**.** *For a literal equation E, the rule*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}$$

is derived, with the usual freshness condition.

Proof. Use recovered progress introduction and QE. □

The new zero representative K matters: the old $I(0,0;u.0)$ has become the syntax 0 and cannot select case mode.

5 Normalization algebra

Lemma 5.1 (Normal forms)**.** *The operation* nf *produces a normal term, is idempotent, and fixes exactly normal terms. It preserves scopes and commutes with variable renaming.*

Proof. Use structural induction. Normalize all iterator children. If its counter is zero, the initializer is already normal and has the surrounding scope; otherwise the rebuilt iterator is normal. Renaming cannot turn a variable, successor, or nonzero-headed iterator into literal zero, so it commutes with the decision. □

The rewrite $I(0,a;u.h) \to a$ terminates because it decreases the number of nodes. A contraction in any position preserves nf: it preserves a retained child's normal form and cannot affect the result when it occurs in an erased body. Every term reduces to its bottom-up normal form. Together these facts prove uniqueness of normal forms, including contractions under binders.

Lemma 5.2 (Substitution coherence)**.** *For every simultaneous raw substitution σ,*

$$\mathrm{nf}(t[\sigma]) = \mathrm{nf}\big(\mathrm{nf}(t)[\,\mathrm{nf}\circ\sigma\,]\big). \tag{NC}$$

The corresponding formula identity holds under both binders.

Proof. Induct on t, with renaming compatibility for lifted substitutions. If an iterator counter has $\mathrm{nf}(c) = 0$, the induction hypothesis implies $\mathrm{nf}(c[\sigma]) = 0$, so both sides return the same substituted initializer. Otherwise substitution may or may not create zero, but both sides use the same smart constructor on the same three inductively equal children. The body uses the lifted substitution. Formula induction uses one lift under Q and two under D. □

Corollary 5.3. *On normal terms, hereditary substitution obeys identity and composition and commutes with weakening and binder instantiation.*

Proof. Apply (NC), idempotence, and the raw substitution laws. □

The normalizer is a structural erasing pass. It does not run successor iteration, test arithmetic equality, or evaluate arbitrary primitive recursive programs. Substitution can enlarge a term by copying its image, as in the baseline; the added normalization removes nodes.

6 The internal round trip

Changing an iterator body is not automatically an instance of ordinary one-hole term congruence. Pointwise equality of bodies must be lifted through iteration.

Lemma 6.1 (Iterator-body congruence)**.** *In the raw native calculus, if $h(u) = k(u)$ is a theorem with u and passive parameters free, then*

$$I(n,a;u.h) = I(n,a;u.k)$$

is a theorem.

Proof. Induct on n using literal-equation induction. Both zero instances equal a. At a successor, the induction hypothesis equates the previous values. Congruence through h, followed by the pointwise theorem instantiated at the second previous value, equates the updates. Successor computation completes the step. The pointwise equation is a theorem, not a retained hypothesis allowed to vary with the iteration state. □

The existing `Q.iter_body_congr` and `Q.iter_congr` in `RoundTripScalar.lean` formalize the ordinary scalar version; the native/scalar bridge transfers it [7, 8].

Theorem 6.2 (Normalization is internally the identity)**.** *For every raw term t, the raw native calculus proves*

$$t = \mathrm{nf}(t). \tag{NI}$$

For every raw formula A, it proves $A \dashv\vdash \mathrm{nf}(A)$, uniformly in its free numerical parameters.

Proof. Induct structurally on t. Variables and zero use reflexivity; successor uses congruence. In $I(c, a; u.h)$, ordinary congruence replaces c, a using their induction hypotheses, and Lemma 6.1 replaces h. If $\mathrm{nf}(c) = 0$, raw zero computation then yields $\mathrm{nf}(a)$; otherwise the term already is $\mathrm{nf}(t)$.

For formulas, use these endpoint equalities at equations and the derived native binder congruences at Q, D. Equivalently, use their ordinary intuitionistic readings and the native/scalar equivalence. The matrix equivalences hold uniformly in the bound variables, so implication and quantifier congruence apply. □

This is an internal identity, not merely equality of standard-model values. Inclusion of normal syntax into raw syntax preserves substitution only up to this proved equality:

$$\mathcal{N} \vdash \mathrm{inc}(t[\sigma]_0) = \mathrm{inc}(t)[\mathrm{inc}\,\sigma].$$

The left term description is definitionally the normal form of the right; the displayed equality is a theorem. The two descriptions need not be identical.

7 Both proof translations

Let $\mathcal{N}$ be the raw eight-constructor native kernel. Normalize formulas by normalizing their terms and retaining their formula nodes.

Theorem 7.1. *If $\Gamma \vdash_{\mathcal{N}} A$, then $\mathrm{nf}(\Gamma) \vdash_{\mathcal{Z}_7} \mathrm{nf}(A)$, hence also $\mathrm{nf}(\Gamma) \vdash_{\mathcal{Z}_6} \mathrm{nf}(A)$.*

Proof. Translate the finite proof tree inductively. Hypothesis membership is preserved. A zero-computation leaf becomes reflexivity, supplied by Lemma 4.1. A successor-computation leaf becomes IS by Lemma 5.2. Replace two-template transport by two X steps. QE, DE, and both introductions commute with normalization by (NC) and its lifted forms. Progress matrices remain equations. Normalization may erase free-variable occurrences but never introduces them, so eigenvariable conditions are preserved. Finally apply Theorem 4.2. □

Do not normalize an old single-introduction proof and blindly retain its introduction mode: a nonliteral witness can become literal zero. The translation passes through the two-introduction systems.

Theorem 7.2. *If $\Gamma \vdash_{\mathcal{Z}_7} A$, inclusion into raw syntax gives $\Gamma \vdash_{\mathcal{N}} A$. The same holds for $\mathcal{Z}_6$.*

Proof. First use Theorem 4.2 if necessary, then induct on the seven-rule proof. Hyp is unchanged. IS is raw successor computation followed by replacing its endpoint with its proved normal form. For X, use the native equalities between raw $h(s), h(t)$ and their hereditary instances, supplied by Theorem 6.2, at both ends of native one-context transport.

For QE, convert the normalized base to its raw instance, use native QE, then convert the target to its normal instance. For DE, the normal matrix itself is unchanged and renaming commutes with normalization, so the fresh local closure has the right reading. For case introduction, convert its raw local assumption $B(t, 0)$ to its normal instance, cut in the translated premise, convert the endpoint back to raw $B(t, Sz)$, then use native case introduction. For progress, use the corresponding coherence at the equation instances and native equation progress. All conversions are parameter-uniform; no eigenvariable enters the retained context or external conclusion. □

Corollary 7.3 (Faithfulness and composites)**.** *For every raw sequent,*

$$\Gamma \vdash_{\mathcal{N}} A \quad \Longleftrightarrow \quad \mathrm{nf}(\Gamma) \vdash_{\mathcal{Z}_6} \mathrm{nf}(A).$$

Normal syntax followed by inclusion and normalization returns literally itself. Raw syntax followed by normalization and inclusion returns a formula internally equivalent to the original.

Proof. Use the two proof translations, Theorem 6.2, and cut. Idempotence proves the first composite and internal normalization correctness proves the other. □

8 Bi-interpretability with full iPRA

Here iPRA is full first-order intuitionistic primitive recursive arithmetic: a symbol for each primitive recursive definition, its defining equations, successor separation and injectivity, and quantifier-free induction, in intuitionistic first-order logic with equality. Only the induction schema is restricted to quantifier-free formulas; the language of theorems is unrestricted.

The pinned repository proves the full native/iPRA bi-interpretation, including both formula round trips, rather than only equality of quantifier-free consequences [8]. Its intermediate theory T_{eq} has intuitionistic first-order logic over iterator terms, the two computation equations, numerical coverage, $0 = S0 \to \bot$, and equation induction. The inherited arithmetic development derives predecessor, reflection of equality, quantifier-free induction, pairing, and a compiler for primitive recursion. The frozen Lean source chain is included.

Let $J : \mathcal{N} \to \mathsf{iPRA}$ and $H : \mathsf{iPRA} \to \mathcal{N}$ be those interpretations. Define

$$J_0 = J \circ \mathrm{inc}, \qquad H_0 = \mathrm{nf} \circ H.$$

Both use the whole numerical domain and actual equality. No quotient of numbers, restricted cut, or coded subclass replaces the domain; only term descriptions are normalized.

Theorem 8.1 (Main theorem)**.** *$\mathcal{Z}_6$ is bi-interpretable and equiconsistent with* iPRA*. Both numerical composites have the identity graph $x = y$.*

Proof. Normalization and inclusion preserve and reflect consequence by Section 7, and their composites have the internal identities of Corollary 7.3. Compose with the pinned native/iPRA round trips. For every normal formula A and arbitrary iPRA formula φ, this yields

$$\mathcal{Z}_6 \vdash A \leftrightarrow H_0 J_0 A, \qquad \mathsf{iPRA} \vdash \varphi \leftrightarrow J_0 H_0 \varphi,$$

where the first equivalence uses the derived native macros. Variables are unchanged, so the numerical identity graph is $x = y$.

Normalization fixes $0, S0$. The native bridge identifies $0 = S0$ with contradiction up to derivability. Thus

$$\mathcal{Z}_6 \vdash 0 = S0 \quad \Longleftrightarrow \quad \mathsf{iPRA} \vdash \bot,$$

which proves equiconsistency. □

8.1 Explicit logical macros

All constructions in this subsection use smart iteration and hereditary substitution. Put $C(v, a, b) = \widehat{I}(v, a; u.b)$, with u fresh. Define an equation-to-formula selector $L(v, s, t; B)$:

$$\begin{aligned} L(v, s, t; r = q) &= (C(v, s, r) = C(v, t, q)), \\ L(v, s, t; \mathsf{Q}_y B) &= \mathsf{Q}_y L(v, C(y, 0, s), C(y, 0, t); B), \\ L(v, s, t; \mathsf{D}_{x,y} B) &= \mathsf{D}_{x,y} L(v, C(y, 0, s), C(y, 0, t); B). \end{aligned}$$

In every binder clause passive arguments are weakened into its scope and bound names are chosen fresh. Set $G(v, A) = L(v, 0, 0; A)$, then define Sel recursively on its first formula:

$$\begin{aligned} \mathrm{Sel}(v, s = t, B) &= L(v, s, t; B), \\ \mathrm{Sel}(v, \mathsf{Q}_y A, B) &= \mathsf{Q}_y \, \mathrm{Sel}(v, A, G(y, B)), \\ \mathrm{Sel}(v, \mathsf{D}_{x,y} A, B) &= \mathsf{D}_{x,y} \, \mathrm{Sel}(v, A, G(y, B)). \end{aligned}$$

These are finite recursive macros, not new primitive connectives. The inherited `NativeGate.lean` proves the endpoint properties, which transfer through normalization:

$$\mathrm{Sel}(0, A, B) \dashv\vdash A, \qquad \mathrm{Sel}(Sz, A, B) \dashv\vdash B, \qquad \vdash G(0, A), \quad G(Sz, A) \dashv\vdash A.$$

Every auxiliary bound variable below is fresh for the input formulas and passive parameters, except for the variable intentionally bound by the displayed existential or universal quantifier. With $p(y) = \widehat{I}(y, 0; u.C(u, 1, 0))$, define

$$\begin{aligned} \bot &:= (0 = S0), \\ A \to B &:= \mathsf{Q}_y \, \mathrm{Sel}(y, A, B), \\ A \wedge B &:= \mathsf{Q}_y G(y, \mathrm{Sel}(p(y), A, B)), \\ \exists x \, A(x) &:= \mathsf{D}_{x,y} G(y, A(x)), \\ A \vee B &:= \exists v \ \mathrm{Sel}(v, A, B), \\ \forall y \, A(y) &:= A(0) \wedge \mathsf{Q}_y A(y). \end{aligned}$$

The inherited `NativeLogic.lean` derives all intuitionistic natural-deduction rules for these macros. Excluded middle for arbitrary formulas is not added. The result is full intuitionistic arithmetic, not only an equational fragment.

9 The standalone checker

9.1 Strict external syntax

The main checker is `code/zero_six.py`. It uses tagged JSON and scoped de Bruijn indices:

Syntax	JSON
variable i	`["v",i]`
0	`["z"]`
$S(t)$	`["s",t]`
$I(c, a; u.h)$	`["i",c,a,h]`, with $c \not\equiv 0$
$s = t$	`["=",s,t]`
$\mathsf{Q}_y B$	`["q",B]`
$\mathsf{D}_{x,y} B$	`["d",B]`

A certificate has exactly four keys: `variables`, `context`, `goal`, `proof`. It specifies the number of free numerical variables, assumptions, claimed formula, and finite proof tree. No executable code is accepted inside a certificate.

The six proof tags are `hyp`, `succ`, `tr`, `qe`, `de`, `di`. The `tr` tag is X. The checker infers each node's conclusion and compares the final result with an independently validated goal. Every introduced term and matrix is checked at the correct scope.

9.2 The charged normalization

The inherited traversal already keeps the iterator counter and initializer at their surrounding scope and binds one variable only in the body. After rebuilding a node, it now performs

```
return a[2] if a[:2] == ['i', ['z']] else a
```

The public term validator additionally requires `c!=['z']` at each iterator. The `zero` proof branch is removed.

External syntax is validated before use. An out-of-scope variable hidden in a supplied $I(0, a; u.h)$ cannot be erased to evade validation: the noncanonical node is rejected. The legacy proof compiler is untrusted and validates its source certificate before normalization. Native canonical certificates require no compiler.

9.3 Soundness, completeness, and resources

A valid finite six-rule proof tree passes the corresponding structural checks. Conversely, each successful inference tag exhibits an instance of its displayed rule, while scope and sort invariants are preserved by hereditary substitution. Induction on the accepted tree therefore gives a derivation in $\mathcal{Z}_6$.

As in the baseline, this is the mathematical unbounded-resource contract. A concrete Python process may reject sufficiently large valid certificates through recursion or memory limits. These limits do not create an acceptance shortcut. Explicit `require` checks, rather than Python `assert`, keep validation active under `python3-O`.

10 Size comparison and limits

The protocol is CPython 3.12.14: parse the full standalone source, apply `ast.unparse`, append a newline, count UTF-8 bytes and significant tokens, and count nodes from `ast.walk`. Gzip uses level 9, fixed zero timestamp, and no filename, and is informational.

Complete tagged checker	Rules	Tokens	AST	Bytes	Gzip-9
Raw, two introductions	8	1,084	1,141	3,322	984
Normal, two introductions	7	1,073	1,136	3,272	996
Raw, single introduction	7	1,024	1,080	3,145	964
Normal, single introduction	6	1,013	1,075	3,095	977

The conservative comparison is the equally factored raw seven-rule control. The final six-rule checker saves 50 bytes (about 1.59%), 11 tokens, and five AST nodes. The two-introduction normal checker also saves 50 bytes against its matched eight-rule raw control. Gzip size increases; it is reported rather than minimized.

All normal-form validation, smart reconstruction, logical rules, scope handling, JSON parsing, final goal comparison, and process accept/reject code are charged. Translators and test runners are untrusted and unnecessary for verifying native canonical certificates.

10.1 A rejected apparent saving

We also examined common-source congruence:

$$\frac{s = t \qquad s = u}{h(t) = h(u)}.$$

It is equivalent to ordinary equality transport. An apparent 36-byte saving disappeared after identical factoring of the existing one-context control: both measured 3,322 bytes, 1,084 tokens, and 1,141 AST nodes. That refactoring is not counted as a mathematical improvement. The final system retains the existing one-context rule.

10.2 The separate compact frontier

The 521-byte arity-coded checker has a different representation, goal contract, and successful-trace argument, with guarded public formulas and auxiliary resources. The tagged checker explicitly validates all term and formula constructors. No reduction of the 521-byte architecture is established here. An absolute minimum would additionally require a fixed description language and an optimality proof; this report supplies neither.

11 Verification and formalization scope

The recorded run has **500 process checks**: 400 expected acceptances and 100 expected rejections, with **zero mismatches**, including runs with ordinary Python and with `-O`. All 64 inherited positive certificates translate successfully to both normal calculi.

The syntactic suite records 4,832 substitution-commutation checks; 462 checks each of idempotence, canonical scope preservation and shift commutation; and 400 checks each of zero erasure, successor computation preservation and formula-root preservation. The exact inputs, tests and results are included in `code/verify.py` and `evidence/verification.json`.

The suite includes genuine equation induction, QE and DE, compound introduction, transport under binders, and the missing atomic case introduction at literal-zero witness. That last case exercises $K = I(S0, 0; u.0)$. A further test uses the replaced local assumption under a nested DE eigenvariable, checking capture-avoiding weakening.

Negative cases address mismatched equality instances, forbidden compound progress, escaping variables, malformed tags, scope errors, and noncanonical iterators in goals, contexts, bodies, templates, seeds, and counters.

The new 255-line `formal/ZeroNormalization.lean` compiles with Lean 4.32.0 and Mathlib revision `81a5d257c8e410db227a6665ed08f64fea08e997`. It proves:

- normal-form characterization, idempotence, renaming and lifting;
- the full term and formula substitution-coherence laws;
- hereditary substitution composition;
- $T_{\mathrm{eq}} \vdash t = \mathrm{nf}(t)$, using iterator-body congruence;
- the ordinary formula equivalence and both raw native formula return derivations, including Q and D.

Its substitution and ordinary correctness theorems depend only on Lean's `propext` and `Quot.sound`. The native formula return theorem additionally inherits `Classical.choice` from the existing native completeness proof. There is no `sorryAx` or added axiom. These are metatheoretic proof-assistant dependencies; the object logic remains intuitionistic.

The pinned native/iPRA dependency chain was rebuilt. The existing one-context modules were also checked against that baseline and are included as attributed references. One inherited namespace reference, `WPrf.eqSymm`, was corrected to `WKernel.eqSymm`; the exact compatibility patch is recorded separately and the frozen source is retained.

Formalization boundary. The new six-rule derivability relation, its entire proof-tree equivalence, and the Python checker are not asserted to be formally verified in Lean. They have the explicit written proof above and executable proof-translation tests. Lean verifies the new normalization and return bridge; the inherited compiled sources verify the iPRA bridge. Tests alone are not used as a universal equivalence proof.

12 Reproduction and contents

Location	Purpose
`paper/report.tex`, `paper/report.pdf`	Complete report and compiled version.
`code/zero_six.py`	Main standalone checker.
`code/zero_seven.py`	Two-introduction bridge checker.
`code/`	Matched controls, translators, fixtures, examples, and verification.
`formal/ZeroNormalization.lean`	Compiled normalization and internal return bridge.
`reference/ConciseIPRA/`	Frozen Lean dependencies and one-context modules.
`reference/prior/`	Explicit antecedents for definitional zero computation.
`evidence/`	Metrics, verification, compilation records, provenance and checksums.

The README gives exact commands. Python checks use only the standard library. LaTeX uses ordinary packages. Lean reproduction requires the stated compiler and pinned Mathlib.

A further step would formalize the normalized proof relation and both proof translations, then examine whether the reduction survives a fully charged adaptation of the much smaller arity codec. The delivered result is the explicit six-rule theory and measured direct-checker reduction.

A The inherited iPRA interface

For each fixed primitive recursive definition, iPRA has a function symbol. Initial functions are zero, successor, and projections. Composition introduces $f(\vec{x}) = g(h_1(\vec{x}), \ldots, h_k(\vec{x}))$. Primitive recursion introduces

$$f(\vec{x}, 0) = g(\vec{x}), \qquad f(\vec{x}, Sn) = h(\vec{x}, n, f(\vec{x}, n)).$$

Successor injectivity, $Sx = 0 \to \bot$, and quantifier-free induction are added to intuitionistic first-order logic with equality.

The native-to-iPRA interpretation compiles iteration to primitive recursion whose step ignores the counter. The reverse scalar interpretation uses pairing to carry counter and state and thereby simulates arbitrary primitive recursion with unary-state iteration. The inherited bridge proves both composites for arbitrary formulas. This report changes neither that compiler nor its numerical domain.

B A scope example

The raw term

$$I(n, a; u.I(0, u; v.0))$$

normalizes to $I(n, a; u.u)$ if the outer counter is not literally zero. In the tagged syntax the inner initializer u was never under the inner binder v; it retains the correct index after erasure. This avoids the dummy-slot lowering cost in the older arity codec.

By contrast $I(S0, 0; u.0)$ does not normalize to zero. Its equality to zero is a theorem of IS. Keeping this distinction enables the single-introduction argument.

References

[1] Vladimir Reshetnikov, *ProveIt: Concise presentations of intuitionistic primitive recursive arithmetic*, commit `ecb3b847290c33b1ef7f04571cac63e97c73a54e`. `https://github.com/VladimirReshetnikov/ProveIt/tree/ecb3b847290c33b1ef7f04571cac63e97c73a54e/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`.

[2] Same repository and commit, `Lean/ConciseIPRA/Native.lean`, `NativeEq.lean`, `NativeGate.lean`, `NativeLogic.lean`. Native rules, equality, selectors and logical macros.

[3] Same, `Lean/ConciseIPRA/NativeOneIntro.lean`. The witness-selected introduction and `wprf_iff_nprf`.

[4] Same, `Research/one_context/`. The crossed one-context rule, tagged checkers, and Lean comparison modules.

[5] Same, `Research/prevalidated_elimination/theory/research.tex`, subsection *Making zero computation definitional*.

[6] Same, `Research/uniform_zero/report.tex`, question on definitional zero computation with a different binder layout.

[7] Same, `Lean/ConciseIPRA/RoundTripScalar.lean`, `Q.iter_body_congr` and `Q.iter_congr`.

[8] Same, `Lean/ConciseIPRA/NativeBiInterpretation.lean` and dependency chain, including `BiInterpretation.lean`. Both interpretations, formula round trips and equiconsistency.

[9] Same, `Research/kernel_refinement/` and `Research/compact_update/`. The 521-byte arity-coded frontier and its stated contract.

[10] This package, `code/`, `formal/`, `evidence/`. Reproducible metrics, translations, tests, and compiled normalization.